

EJERCICIOS DE INTRODUCCIÓN A PYTHON

Algoritmos y Estructuras de Datos

Objetivo. Practicar contenidos de Python básico: variables, tipos, operadores y funciones; colecciones y control de flujo; y errores, excepciones y validación de entradas. Los ejercicios no requieren programación orientada a objetos.

ENTREGA 1 — Datos, cálculos y decisiones

Cada ejercicio se califica sobre 10 puntos. La rúbrica indica qué se espera y cómo se repartirá la puntuación.

Ejercicio 1. El termómetro inteligente

Vamos a construir un programa muy pequeño que reciba una temperatura y nos diga dos cosas: cuánto equivale en grados Fahrenheit y cómo podemos clasificar esa temperatura.

Qué tienes que hacer

1. Pide al usuario una temperatura en grados Celsius. Puede tener decimales.
2. Convierte el dato introducido a un número de tipo float.
3. Calcula los grados Fahrenheit usando la fórmula: $F = C \times 9/5 + 32$.
4. Clasifica la temperatura en uno de estos tres grupos: menos de 10 °C → “frío”; de 10 °C a menos de 25 °C → “temperatura moderada”; 25 °C o más → “calor”.
5. Muestra en pantalla la temperatura original, la temperatura en Fahrenheit y la clasificación.
6. Si el usuario escribe algo que no sea un número, evita que el programa se rompa y muestra un mensaje claro.

Conceptos principales que debes utilizar: `input()` y `print()` · `float()` · variables · operadores aritméticos · `if` / `elif` / `else` · `try` / `except ValueError` · f-strings

Comprobación rápida: Si el usuario introduce 20, el programa debe indicar aproximadamente 68 °F y “temperatura moderada”.

Ejercicio 2. La semana de temperaturas

Ahora trabajaremos con varios datos a la vez. Tendrás una lista con las temperaturas máximas registradas durante siete días y deberás obtener información útil de ella.

Qué tienes que hacer

1. Parte de esta lista: `temperaturas = [18, 21, 19, 24, 27, 25, 20]`.
2. Calcula la temperatura máxima y la mínima.
3. Calcula la temperatura media de la semana. Para practicar bucles, haz el cálculo de la media recorriendo la lista y acumulando la suma.

4. Cuenta cuántos días tuvieron una temperatura superior a la media.
5. Crea una lista nueva que contenga solo las temperaturas superiores a la media.
6. Muestra todos los resultados de forma clara.

Conceptos principales que debes utilizar: listas · for · acumuladores · len() · if · append() · min() y max()

Comprobación rápida: La media debe calcularse a partir de los siete valores, no escribirse manualmente.

Ejercicio 3. ¿Cuánta electricidad hemos gastado?

Una vivienda registra el consumo eléctrico de varios días. Queremos calcular cuánto ha consumido y cuánto costaría la energía utilizada.

Qué tienes que hacer

1. Usa esta lista de consumos en kWh: consumos = [8.2, 7.5, 9.1, 12.0, 6.8, 10.4, 11.2].
2. Guarda en una variable el precio de la electricidad: 0.18 euros por kWh.
3. Calcula el consumo total y el consumo medio diario.
4. Calcula el coste total de la energía consumida.
5. Considera “consumo alto” cualquier día con más de 10 kWh. Muestra qué días de la semana tienen consumo alto. Puedes numerar los días del 1 al 7.
6. Muestra un resumen final con total, media, coste y días de consumo alto.

Conceptos principales que debes utilizar: listas · for · enumerate() · acumuladores · if · operadores · f-strings

Comprobación rápida: El día 4 tiene 12.0 kWh, por lo que debe aparecer como día de consumo alto.

Ejercicio 4. Una función que analiza cualquier lista de datos

Hasta ahora hemos repetido cálculos parecidos. Vamos a ponerles nombre y guardarlos dentro de una función para poder reutilizarlos.

Qué tienes que hacer

1. Crea una función llamada analizar_datos(datos).
2. La función recibirá una lista de números.
3. Debe calcular la media, el valor mínimo y el valor máximo.
4. Debe devolver los tres resultados. Puedes devolverlos como una tupla: media, minimo, maximo.
5. Prueba la función con al menos dos listas diferentes.

6. Añade una comprobación para evitar dividir entre cero si la lista está vacía. En ese caso, la función debe lanzar `ValueError` con un mensaje claro.

7. Incluye al menos dos `assert` para comprobar que tu función devuelve resultados correctos.

Conceptos principales que debes utilizar: `def` · parámetros · `return` · listas · tuplas / retorno múltiple · `raise` `ValueError` · `assert`

Comprobación rápida: `analizar_datos([2, 4, 6])` debería devolver media 4, mínimo 2 y máximo 6.

Ejercicio 5. Calculadora segura

Vas a crear una calculadora de consola. El objetivo no es solo calcular, sino conseguir que el programa no se rompa si el usuario se equivoca.

Qué tienes que hacer

1. Crea una función `leer_numero(mensaje)` que pida un número y repita la pregunta hasta recibir un valor válido.
2. Muestra un pequeño menú con cuatro operaciones: `+`, `-`, `*` y `/`.
3. Lee dos números usando la función anterior.
4. Realiza la operación elegida.
5. Si el usuario intenta dividir entre cero, muestra un mensaje claro sin terminar con un error de Python.
6. Si la operación introducida no es una de las cuatro permitidas, indícalo.
7. Organiza el código en funciones siempre que te ayude a evitar repeticiones.

Conceptos principales que debes utilizar: funciones · `while` · `input()` · `try / except` `ValueError` · `ZeroDivisionError` · `if / elif / else` · `return`

Comprobación rápida: Si se elige `/` y el segundo número es 0, el programa debe decir que no se puede dividir entre cero.